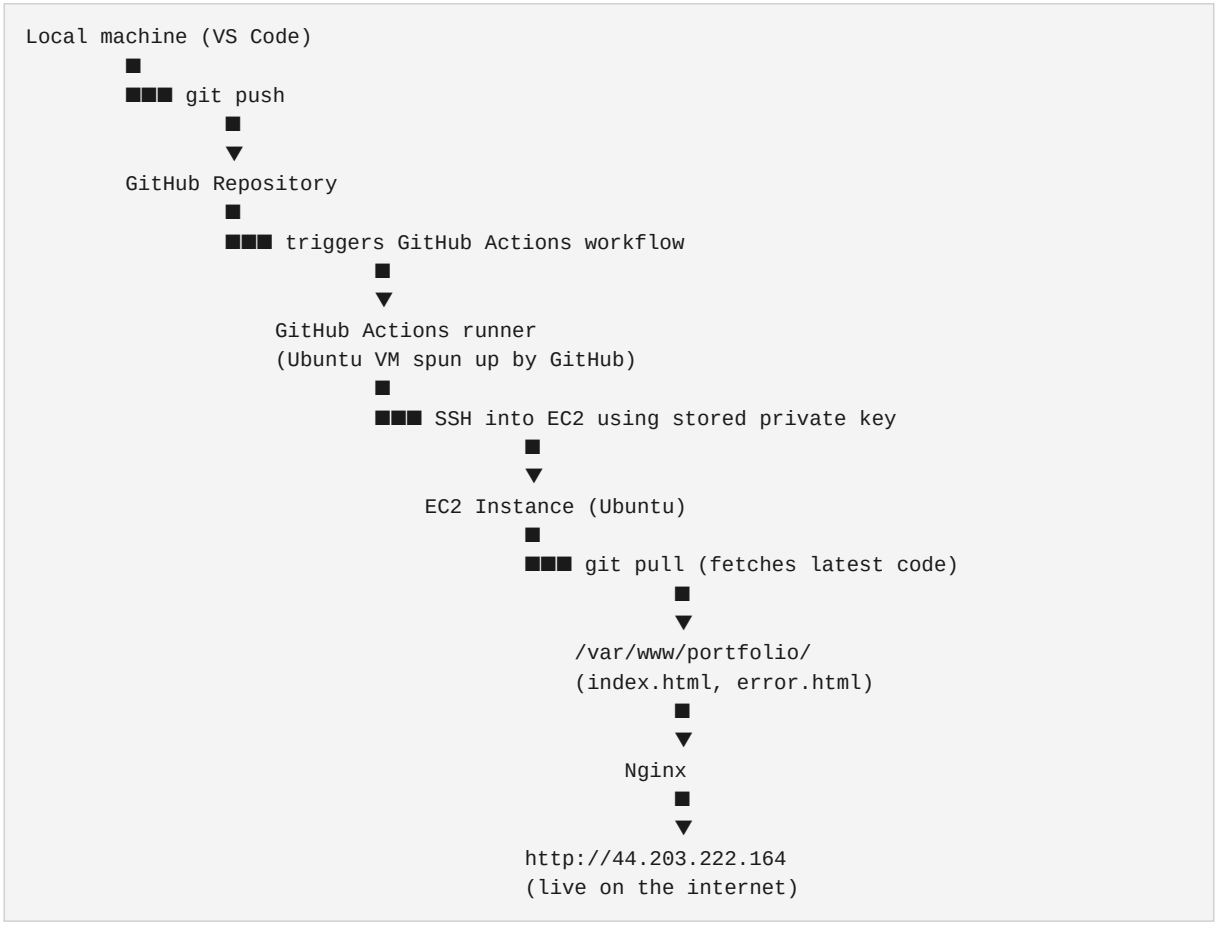
EC2 Nginx Auto-Deploy — Complete Project Notes

Personal reference document. Every command we ran, what it does, and why we did it.

What We Built

A static portfolio website hosted on an AWS EC2 instance, served by Nginx, with an automated CI/CD pipeline using GitHub Actions. Every time we push code to the `main` branch on GitHub, Actions automatically SSHs into the EC2 and pulls the latest code — and Nginx serves it instantly.

Full Flow (Big Picture)



Part 1 — Launching the EC2 Instance (AWS Console)

What we did

Launched a virtual Linux server on AWS that runs 24/7 and is accessible over the internet.

Settings used

- **AMI:** Ubuntu 22.04 LTS — a popular, stable Linux distribution. Free tier eligible.
- **Instance type:** t2.micro — 1 vCPU, 1GB RAM. Free tier eligible.
- **Key pair:** Created a new key pair called `pc-keypair`. Downloaded the `.pem` file to local machine. This is the private key used to SSH into the instance.
- **Security Group inbound rules:**

Type	Port	Source	Why
SSH	22	My IP	Allows us to connect to the server via terminal

HTTP	80	0.0.0.0/0	Allows anyone on the internet to access the website

Key concepts

- **EC2 (Elastic Compute Cloud):** A virtual machine running on AWS infrastructure. Unlike S3 static hosting (which is managed by AWS), EC2 gives you a full Linux server you control completely.
- **Security Group:** Acts as a firewall. Controls what traffic is allowed in (inbound) and out (outbound) of your EC2. Port 22 = SSH, Port 80 = HTTP web traffic.
- **Key pair:** AWS generates a public/private key pair. AWS keeps the public key on the EC2. You keep the `.pem` file (private key). When you SSH in, your private key is matched against the public key to authenticate you — no password needed.
- **Public IPv4:** `44.203.222.164` — this is the address of our server on the internet.

Part 2 — Connecting to EC2

What we did

Connected to the EC2 instance directly from the AWS console using EC2 Instance Connect.

How SSH normally works (local machine)

If connecting from a local terminal, you first fix the `.pem` file permissions:

```
chmod 400 pc-keypair.pem
```

Why: SSH refuses to use a private key file if it's readable by others. `chmod 400` makes the file readable only by you (owner). Without this, SSH throws a "permissions are too open" error.

Then connect:

```
ssh -i pc-keypair.pem ubuntu@44.203.222.164
```

- `-i pc-keypair.pem` — specifies which private key to use
- `ubuntu` — the default username for Ubuntu EC2 instances
- `44.203.222.164` — the public IP of our EC2

What we actually used

EC2 Instance Connect — a browser-based terminal built into the AWS console. Does the same thing as SSH but without needing a local terminal or the `.pem` file. Perfectly fine for setup.

Part 3 — Installing and Configuring Nginx

Step 1 — Update package lists and install Nginx

```
sudo apt update && sudo apt install nginx -y
```

What this does:

- `sudo` — runs the command as superuser (admin). Required for installing software.
- `apt update` — refreshes the list of available packages from Ubuntu's package repositories. Always run this before installing anything so you get the latest versions.
- `apt install nginx -y` — installs Nginx. `-y` automatically answers "yes" to any prompts so it doesn't pause and wait for input.
- `&&` — runs the second command only if the first one succeeds.

What is Nginx: A web server. Its job is to listen for incoming HTTP requests on port 80 and respond by serving files (like `index.html`). Think of it as the software that actually handles "someone visited my website" and sends back the page.

Step 2 — Start Nginx and enable auto-start

```
sudo systemctl start nginx && sudo systemctl enable nginx
```

What this does:

- `systemctl start nginx` — starts the Nginx service right now.
- `systemctl enable nginx` — tells the system to automatically start Nginx every time the EC2 reboots. Without this, Nginx would stop running after a restart and the site would go down.

Step 3 — Verify Nginx is running

```
sudo systemctl status nginx
```

What this does: Shows the current status of the Nginx service. You should see `active (running)` in green — this confirms Nginx is live and serving traffic.

At this point, visiting `http://44.203.222.164` in a browser showed the **default Nginx welcome page** — proof that the web server was working.

Step 4 — Create the website directory

```
sudo mkdir -p /var/www/portfolio
```

What this does:

- `mkdir` — make directory (create a folder).
- `-p` — creates parent directories too if they don't exist. No error if folder already exists.
- `/var/www/portfolio` — this is where our website files (`index.html`, `error.html`) will live. `/var/www/` is the conventional location for website files on Linux servers.

Step 5 — Give ubuntu user ownership of the folder

```
sudo chown -R ubuntu:ubuntu /var/www/portfolio
```

What this does:

- `chown` — change ownership of a file or folder.
- `-R` — recursive, applies to all files and subfolders inside too.
- `ubuntu:ubuntu` — sets both the owner and group to `ubuntu` (the default user on Ubuntu EC2).
- **Why this matters:** By default, `/var/www/` is owned by root. If GitHub Actions tries to write files there as the `ubuntu` user, it would get a "permission denied" error. Changing ownership to `ubuntu` fixes this.

Step 6 — Create Nginx site configuration

```
sudo nano /etc/nginx/sites-available/portfolio
```

What this does: Opens the `nano` text editor to create a new Nginx configuration file called `portfolio`. `/etc/nginx/sites-available/` is where Nginx config files are stored.

Config we pasted:

```
server {
    listen 80;
    server_name 44.203.222.164;

    root /var/www/portfolio;
    index index.html;

    error_page 404 500 502 503 504 /error.html;

    location / {
        try_files $uri $uri/ =404;
    }
}
```

What each line means:

- `listen 80` — Nginx listens for traffic on port 80 (standard HTTP port).
- `server_name 44.203.222.164` — matches requests coming to this IP.
- `root /var/www/portfolio` — tells Nginx where the website files are.
- `index index.html` — the default file to serve when someone visits the root URL.
- `error_page 404 500 502 503 504 /error.html` — for any of these error codes, serve our custom `error.html` instead of Nginx's default error page.
- `location /` — handles all incoming requests.
- `try_files $uri $uri/ =404` — tries to find the requested file, if not found returns 404.

Saved with: `Ctrl+X` → `Y` → `Enter`

Step 7 — Enable the site configuration

```
sudo ln -s /etc/nginx/sites-available/portfolio /etc/nginx/sites-enabled/
```

What this does: Creates a symbolic link (shortcut) from `sites-available` to `sites-enabled`. Nginx only serves sites listed in `sites-enabled`. This is Nginx's way of letting you have config files ready without activating them — you enable them by linking.

Step 8 — Test the Nginx configuration

```
sudo nginx -t
```

What this does: Tests the Nginx config files for syntax errors without actually restarting Nginx. Always run this before reloading — if there's an error in your config, reloading would break the web server.

Output we got:

```
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
```

This means our config is valid.

Step 9 — Remove the default Nginx site

```
sudo rm /etc/nginx/sites-enabled/default
```

What this does: Removes the default Nginx welcome page config from `sites-enabled`. If we don't do this, the default config conflicts with ours and Nginx might serve the wrong thing (the welcome page instead of our portfolio).

Step 10 — Reload Nginx

```
sudo systemctl reload nginx
```

What this does: Reloads Nginx config without fully restarting it. Nginx applies the new configuration gracefully — active connections aren't dropped. After this, Nginx serves from `/var/www/portfolio` instead of the default location.

At this point visiting `http://44.203.222.164` showed a **403 Forbidden** — expected because the `/var/www/portfolio` folder was empty. Nginx was correctly configured but had no files to serve yet.

Part 4 — Setting Up GitHub Actions CI/CD

What we did

Configured automated deployment — every push to `main` triggers GitHub Actions to SSH into EC2 and pull the latest code.

Step 1 — Add GitHub Secrets

Go to GitHub repo → **Settings** → **Secrets and variables** → **Actions** → **New repository secret**

Secret Name	Value	Why
`EC2_HOST`	`44.203.222.164`	The IP address GitHub Actions SSHs into
`EC2_USER`	`ubuntu`	The Linux username to log in as
`EC2_SSH_KEY`	Full contents of `pc-keypair.pem`	The private key for authentication

Why secrets: We never hardcode sensitive values like IPs and private keys directly in workflow files. GitHub Secrets encrypts them and injects them at runtime. The private key especially must never be committed to the repo — anyone with it can access your EC2.

Step 2 — Create the GitHub Actions workflow

File: `.github/workflows/deploy.yml`

```
name: Deploy to EC2

on:
  push:
    branches:
      - main

jobs:
  deploy:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v3

      - name: Deploy to EC2 via SSH
        uses: appleboy/ssh-action@v1.0.0
        with:
          host: ${{ secrets.EC2_HOST }}
          username: ${{ secrets.EC2_USER }}
          key: ${{ secrets.EC2_SSH_KEY }}
          script: |
            cd /var/www/portfolio
            sudo git pull origin main || true
```

What each section means:

- `name: Deploy to EC2` — display name shown in the GitHub Actions UI.
- `on: push: branches: main` — this workflow triggers automatically on every push to the `main` branch.
- `runs-on: ubuntu-latest` — GitHub spins up a fresh Ubuntu virtual machine to run this workflow.
- `actions/checkout@v3` — checks out (downloads) your repo code onto the GitHub Actions runner VM.
- `appleboy/ssh-action@v1.0.0` — a community GitHub Action that handles SSH connections. Takes care of all the SSH setup so we don't have to write it manually.
- `host, username, key` — pulled from GitHub Secrets at runtime.
- `script` — the commands that run on your EC2 after SSH connects:
- `cd /var/www/portfolio` — navigate to the website folder.
- `sudo git pull origin main` — pull the latest code from GitHub into the folder.
- `|| true` — if `git pull` fails for any reason, don't fail the whole workflow. Keeps the pipeline from breaking on minor issues.

Part 5 — Setting Up Git on EC2

What we did

Installed git on EC2 and cloned the repo so `git pull` works during deployments.

Step 1 — Install git on EC2

```
sudo apt install git -y
```

EC2 instances don't come with git pre-installed. We need it so the EC2 can run `git pull` during deployments.

Step 2 — Clone the repo into the portfolio folder

```
cd /var/www/portfolio
sudo git clone https://github.com/Siddhesh-07/ec2-nginx-auto-deploy.git .
```

What this does:

- `cd /var/www/portfolio` — navigate to the website directory.
- `git clone .` — clones the GitHub repo into the current directory. The `.` at the end is important — without it, git would create a subfolder called `ec2-nginx-auto-deploy` inside `/var/www/portfolio`, and Nginx wouldn't find the files.

Step 3 — Fix ownership after clone

```
sudo chown -R ubuntu:ubuntu /var/www/portfolio
```

After cloning with `sudo`, the files are owned by root. We reset ownership to `ubuntu` so future `git pull` commands (which run as `ubuntu` via GitHub Actions) can write files without permission errors.

End Result

What	Where
Live website	`http://44.203.222.164`
Repo	`github.com/Siddhesh-07/ec2-nginx-auto-deploy`
Website files	`/var/www/portfolio/` on EC2
Nginx config	`/etc/nginx/sites-available/portfolio`
Workflow file	`.github/workflows/deploy.yml`

How to Deploy Going Forward

Just push to main:

```
git add .
git commit -m "your message"
git push
```

GitHub Actions handles everything else automatically.

Key Differences vs S3 Project

ject	This Project	
g	AWS managed storage	Self-managed Linux server
erver	S3 built-in	Nginx (installed by us)
ment method	`aws s3 sync`	SSH + `git pull`
cess	None	Full Linux control
files	None	Nginx config, systemd
eshooting	AWS console	Linux commands (`systemctl`, `nginx -t`)

Important File Paths on EC2

Path	What it is
`/var/www/portfolio/`	Website files served by Nginx
`/etc/nginx/sites-available/portfolio`	Our Nginx config file
`/etc/nginx/sites-enabled/portfolio`	Symlink that activates the config
`/etc/nginx/nginx.conf`	Main Nginx config (we didn't edit this)

Useful Commands for Later

```
# Check Nginx status
sudo systemctl status nginx

# Restart Nginx
sudo systemctl restart nginx

# Reload Nginx config (no downtime)
sudo systemctl reload nginx

# Test Nginx config for errors
sudo nginx -t

# View Nginx error logs
sudo tail -f /var/log/nginx/error.log

# View Nginx access logs (see incoming requests)
sudo tail -f /var/log/nginx/access.log

# Check what's in the portfolio folder
ls -la /var/www/portfolio
```

Possible Next Steps

- Add **CloudFront** in front of EC2 for HTTPS and CDN
- Point a custom domain using **Route 53**
- Redo the entire EC2 + Nginx + Security Group setup using **Terraform** (IaC)
- Add a **test stage** to the GitHub Actions workflow before deployment